Kyselyn optimointi

Tehokkaan laskentatavan etsintä

Tietokantaratkaisut
Metropolia Ammattikorkeakoulu
Vesa Ollikainen

Oppimistavoitteet

1. Ymmärretään, mitä SQL-lauseelle tapahtuu, kun TKHJ vastaanottaa sen.
2. Opitaan tarkastelemaan kyselyn suorituksen tehokkuuteen vaikuttavia tekijöitä.

SQL-kyselyn suoritusvaiheet

1. Skanneri eristää kyselystä merkitysyksiköt (*tokens*).
2. Jäsennin tarkistaa syntaksin kelpoisuuden ja rakentaa jäsennyspuun.
3. Validaattori tarkistaa semanttisen kelpoisuuden.
   - Esimerkiksi taulujen ja kenttien nimet ovat olemassa.
4. Kyselyn optimoija etsii kyselylle toteutussuunnitelman (tehokkaan suoritustavan).
5. Koodin generoija luo järjestelmäriippuvan kyselykoodin.
6. Tietokantaprosessori suorittaa järjestelmäriippuvan kyselykoodin.

Deskriptiivisen kyselyn toteutus

- SQL on deskriptiivinen kyselykieli.
  - Kyselyn kirjoittava henkilö täsmentää, mitä tietoa vastaukseen halutaan.
  - Siihen, miten tulos lasketaan, ei periaatteessa oteta kantaa.
- Jäsennin muuntaa SQL-kyselyn jäsennyspuuksi.
  - Jäsennyspuun rakenne määrää laskentajärjestyksen.
  - Luotu jäsennyspuu on alustava sisäinen esitys kyselylle.
  - Kutakin jäsennyspuuta vastaa relaatioalgebran lauseke.
- Tarkastellaan aluksi neljä relaatiomallin määräämää perusoperaatiota: projektio, valinta, karteesinen tulo ja liitos.

Projektio

- Projektio valitsee relaatiosta joukon kenttiä.

TYÖNTEKIJÄ				
ID	Etunimi	Sukunimi	Osasto	Svuosi
1	Ville	Väätäinen	1	1971
2	Henriikka	Karjalainen	2	1982
3	Lyly	Salokorpi	1	1953
4	Ville	Skoglund	3	1960

$$\rho_{ID,Sukunimi}(TYÖNTEKIJÄ)$$

ID	Sukunimi
1	Väätäinen
2	Karjalainen
3	Salokorpi
4	Skoglund

Valinta

- Valinta poimii annetun ehdon täyttävät tietueet.

$$s_{ID=2}(TYÖNTEKIJÄ)$$

ID	Etunimi	Sukunimi	Osasto	Svuosi
2	Henriikka	Karjalainen	2	1982

Karteesinen tulo

- Karteesinen tulo yhdistää kunkin ensimmäisen taulun tietueen kuhunkin toisen taulun tietueeseen.
  - Yleensä ei-toivottu operaatio.

TYÖNTEKIJÄ

ID	Etunimi	Sukunimi	Osasto	Svuosi
1	Ville	Väätäinen	1	1971
2	Henriikka	Karjalainen	2	1982
3	Lyly	Salokorpi	1	1953
4	Ville	Skoglund	3	1960

OSASTO

Numero	Nimi	Budjetti
1	Markkinointi	180000
2	Laskutus	45000
3	Rahoitus	125000
4	Tutkimus	400000

TYÖNTEKIJÄ´ OSASTO

ID	Etunimi	Sukunimi	Osasto	Svuosi	Numero	Nimi	Budjetti
1	Ville	Väätäinen	1	1971	1	Markkinointi	180000
1	Ville	Väätäinen	1	1971	2	Laskutus	45000
1	Ville	Väätäinen	1	1971	3	Rahoitus	125000
1	Ville	Väätäinen	1	1971	4	Tutkimus	400000
2	Henriikka	Karjalainen	2	1982	1	Markkinointi	180000
2	Henriikka	Karjalainen	2	1982	2	Laskutus	45000
2	Henriikka	Karjalainen	2	1982	3	Rahoitus	125000
2	Henriikka	Karjalainen	2	1982	4	Tutkimus	400000
3	Lyly	Salokorpi	1	1953	1	Markkinointi	180000
3	Lyly	Salokorpi	1	1953	2	Laskutus	45000
3	Lyly	Salokorpi	1	1953	3	Rahoitus	125000
3	Lyly	Salokorpi	1	1953	4	Tutkimus	400000
4	Ville	Skoglund	3	1960	1	Markkinointi	180000
4	Ville	Skoglund	3	1960	2	Laskutus	45000
4	Ville	Skoglund	3	1960	3	Rahoitus	125000
4	Ville	Skoglund	3	1960	4	Tutkimus	400000

Liitos

- Liitos yhdistää karteesisen tulon ja valinnan.
  - Yhdistetään liitosattribuutin perusteella.
  - Tiedonhallintaohjelmistoissa on liitoksille tehokkaat algoritmit.
    - Perustuvat RAM-muistissa olevien liitospuskureiden käyttöön.

TYÖNTEKIJÄ

ID	Etunimi	Sukunimi	Osasto	Svuosi
1	Ville	Väätäinen	1	1971
2	Henriikka	Karjalainen	2	1982
3	Lyly	Salokorpi	1	1953
4	Ville	Skoglund	3	1960

OSASTO

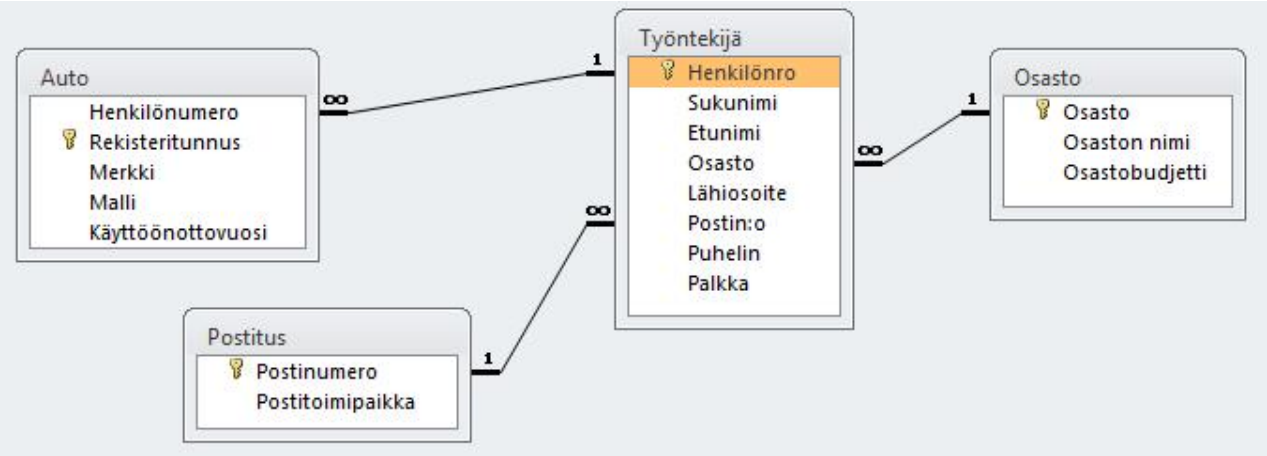
Numero	Nimi	Budjetti
1	Markkinointi	180000
2	Laskutus	45000
3	Rahoitus	125000
4	Tutkimus	400000

$$TYÖNTEKIJÄ >\!\!<_{Osasto=Numero} OSASTO$$

ID	Etunimi	Sukunimi	Osasto	Svuosi	Numero	Nimi	Budjetti
1	Ville	Väätäinen	1	1971	1	Markkinointi	180000
2	Henriikka	Karjalainen	2	1982	2	Laskutus	45000
3	Lyly	Salokorpi	1	1953	1	Markkinointi	180000
4	Ville	Skoglund	3	1960	3	Rahoitus	125000

Esimerkki

- Palataan kyselyn optimointiin. Tarkastellaan eri tapoja suorittaa esimerkkikysely.

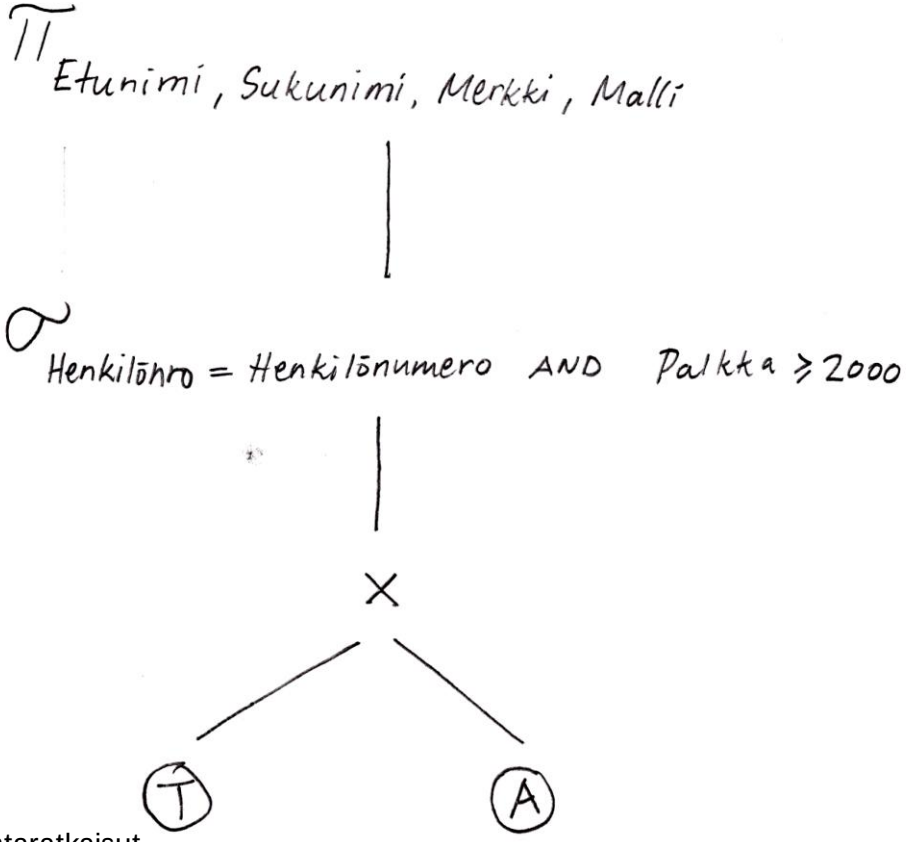



```
SELECT Etunimi, Sukunimi, Palkka, Merkki
FROM Työntekijä, Auto
WHERE Työntekijä.Henkilönro = Auto.Henkilönumero
AND Palkka >= 2000;
```

Alustava jäsennyspuu

```
SELECT Etunimi, Sukunimi, Palkka, Merkki
FROM Työntekijä, Auto
WHERE Työntekijä.Henkilönro = Auto.Henkilönumero
AND Palkka >= 2000;
```

- Oheinen alustava jäsennyspuu on jäsentimen tuottama sisäinen esitys kyselylle.
- Puuta luetaan alhaalta ylös.
  1. Ensin karteesinen tulo TYÖNTEKIJÄ- ja AUTO-taulujen välillä.
  2. Sitten valintaoperaatio.
  3. Lopuksi poimitaan kentät (projektio).
- Sen suoritus on tehotonta.



Kyselyn optimointi

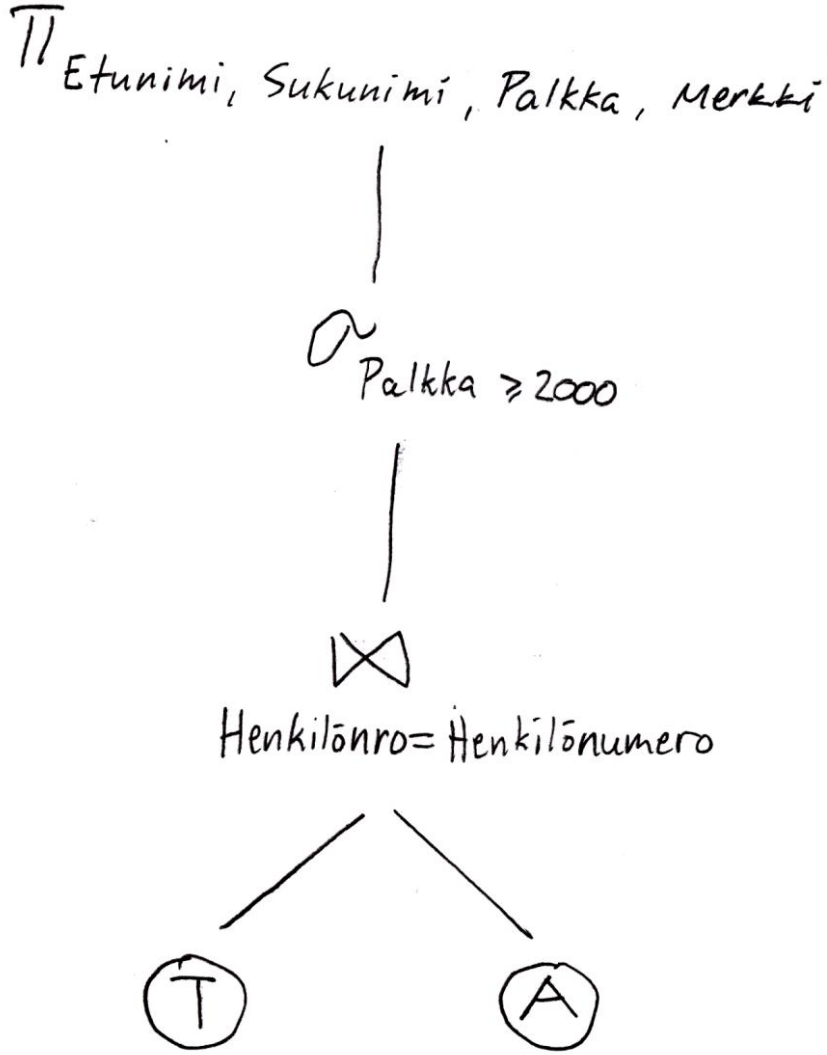
- Miten jäsennyspuu saadaan muokatuksi sellaiseksi, että sitä vastaa tehokkaammin laskettava (laajennetun) relaatioalgebran lauseke?
- Kyselyn optimoija voi soveltaa seuraavia keinoja
  - Heuristiset säännöt
    - Yleispäteviä peukalosääntöjä jäsennyspuun muunnoksille, jotka yleensä nopeuttavat suoritusaikaa havaitun kaltaisissa kyselyissä.
  - Kustannuspohjainen optimointi
    - Luodaan kustannusfunktio, joka kuvaa kirjoitetun kyselyn toteutusaikaa.
    - Arvioidaan, millä tavoin jäsennyspuun muutokset vaikuttavat kustannusfunktioon.
    - Etsitään sellaiset jäsennyspuun muunnokset, jotka minimoivat kustannusfunktion.
  - Semanttinen optimointi
    - Hyödynnetään tietokannan rajoitteita kyselyn korvaamiseksi yksinkertaisemmalla.
    - Täydentävässä roolissa heuristiikoille ja kustannuspohjaiselle optimoinnille.

Kyselyn optimoinnin tavoite

- Tavoitteena on löytää kyselylle toteutussuunnitelma, joka on riittävän tehokas.
- Nopeimman toteutussuunnitelman löytäminen ei yleensä ole mahdollista.
  - Kaikkien suunnitelmien joukko on liian suuri pienimpiä tapauksia lukuun ottamatta.
  - Optimointi perustuu arvioihin suoritusajasta eikä tarkkaan tietoon.
- Optimoinnilla saatavan hyödyn on oltava riittävä suhteessa sen vaatimaan aikaan.
  - Tämä korostuu, jos optimointi tehdään joka kerta ennen kuin kysely suoritetaan (kyselyn tulkkaus vs. kääntäminen).

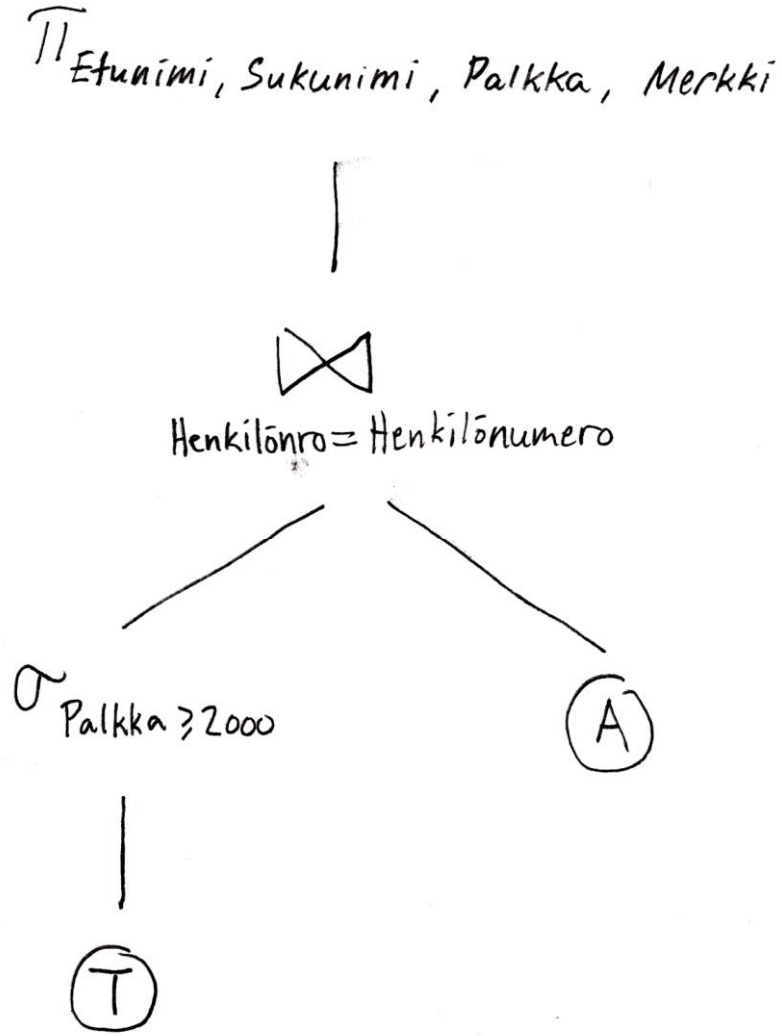
Esimerkki heuristisista säännöistä

- Oheinen jäsennyspuu tuottaa saman tuloksen kuin alustava jäsennyspuu.
- Karteesinen tulo on korvattu luonnollisella liitoksella.
  - Muistiin ja/tai levylle tallennettavat välitulokset ovat pienempiä.
  - Kyselyn suoritus on todennäköisesti nopeampi.



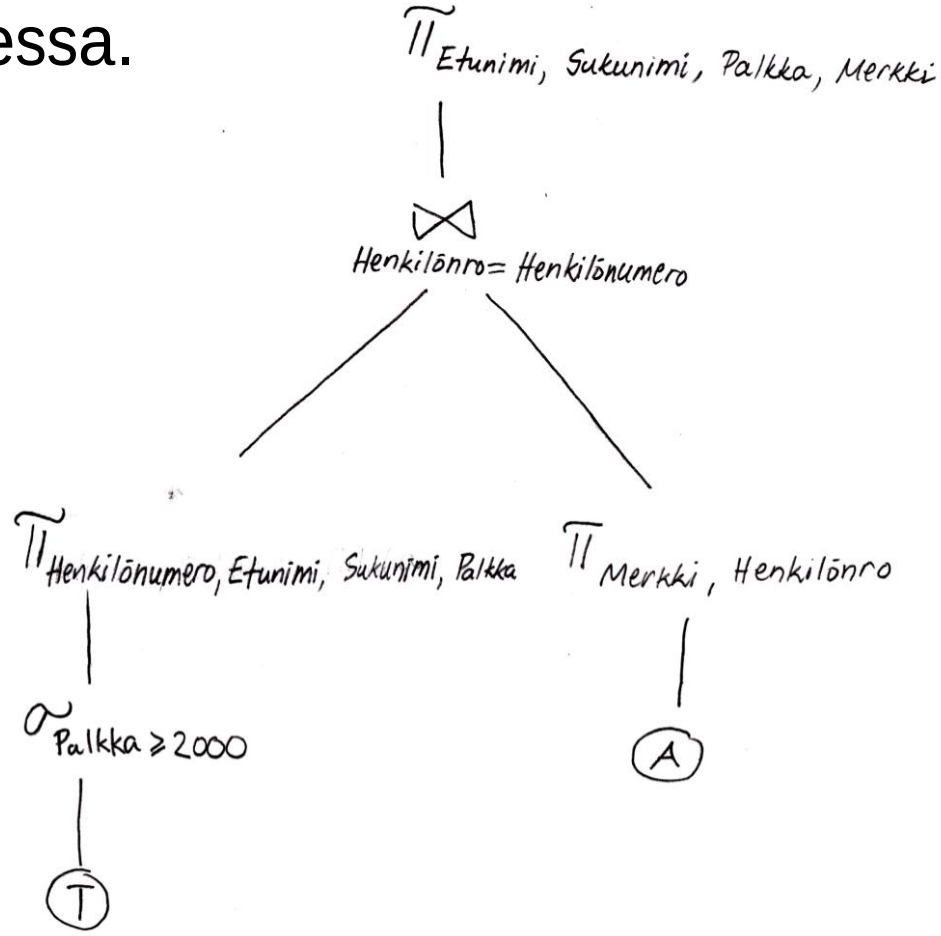
Esimerkki heuristisista säännöistä

- Myös seuraava jäsennyspuu tuottaa identtisen tuloksen.
- Varhaisessa vaiheessa tehtävä valintaoperaatio pienentää välitulosta entisestään.
  - Suoritusaika putoaa oletettavasti edelleen.



Esimerkki heuristisista säännöistä

- Välitulokset pienenevät vielä suorittamalla projektiot varhaisemmassa vaiheessa.



Heuristiikat pähkinänkuoressa

- Suoritetaan mahdollisimman varhaisessa vaiheessa operaatiot, jotka pienentävät välitulosten kokoa.
  - Valinta pienentää monikoiden (rivien) määrää.
  - Projektio pienentää attribuuttien (kenttien) määrää.
- Valinta- ja liitosoperaatioista suoritetaan ensin ne, jotka pienentävät välitulosta tehokkaimmin.

Vaihtoehto: kustannuspohjainen optimointi

- Kyselyn optimoijan laskema kustannusfunktio voi huomioida seuraavia kustannuskomponentteja:
  - Levyhaut
  - Välitulosten tallennus levylle
  - Tuloksen laskenta-aika keskusmuistissa
  - Keskusmuistin käyttö (varaaminen)
  - Kommunikaatio (tuloksen lähetys)
- Dominoiva komponentti voi vaihdella
  - Suuressa tietokannassa levyhaut
  - Pienessä tietokannassa keskusmuistilaskenta
  - Hajautetussa tietokannassa kommunikaatio
- Levyhakujen kustannus riippuu muun muassa kunkin taulun koosta, indeksoinnista ja osituksesta.

Lähde: Elmasri, Navathe, Fundamentals of Database Systems, 2004.

Vaihtoehto: semanttinen kyselyn optimointi

- Tarkistetaan tietokannassa vallitsevia ehtoja ja eheysrajoitteita.
  - Tämän optimointitavan merkitys on yleensä marginaalinen; sitä voidaan soveltaa harvoin.
- Esimerkki semanttisen optimoinnin ideasta:
  - Kysely, joka käskee hakea sellaiset esimies-alainen-parit työntekijätaulusta, joille alaisen palkka on suurempi kuin esimiehen.
  - Tietokannassa on voimassa ehto, että alaisen palkka ei voi olla korkeampi kuin esimiehen.
  - Semanttinen optimoija havaitsee ehdon. Kyselyä ei tarvitse suorittaa, ja voidaan palauttaa tyhjä tulostaulu.

Esimerkin lähde: Elmasri, Navathe, Fundamentals of Database Systems, 2004.

Optimointi fyysisessä tietokantasuunnittelussa

- Tietokantasuunnittelijan optimointi perustuu datan ja kyselyjen analyysiin, suoritusaikamittauksiin ja simulointeihin.
- Keinoja
  - Indeksointi
    - Indeksien tarpeen selvitys
    - Indeksien käyttöasteen ja vaikutuksen analysointi
  - Denormalisointi
    - Luodaan tietokantaan tahallista toisteisuutta tehokkuussyistä.
    - Palataan joidenkin taulujen osalta alempiin normaalimuotoihin.
  - Taulurakenteen muutos
    - Korvataan taulurakenne hakujen ja päivityksen kannalta tehokkaammalla mutta teoreettisesti yhtä hyvällä rakenteella.
  - Osiointi
    - Jaetaan taulut fyysisesti useaan osaan, jolloin keskeiset haut voidaan ehkä kohdistaa esimerkiksi yhteen osaan kerrallaan.
  - Kyselyn syntaktinen optimointi
    - Muokataan kyselyjä manuaalisesti tilanteissa, joissa kyselyn optimoija ei osaa toimia hyvin.

Lähde: Elmasri, Navathe, Fundamentals of Database Systems, 2004.

Esimerkki optimoinnista: ositus

- Osituksessa hyvin suuri tietokantataulu jaetaan erillisiin fyysisiin osioihin.
- Osiot voivat sijaita eri tallennuslaitteilla tai eri palvelimilla.
- Esimerkki osiointimäärityksestä, joka kirjoitetaan **CREATE TABLE** -lauseen loppuun:

```
PARTITION BY RANGE (Palkka) (
           PARTITION p0 VALUES LESS THAN (2000),
           PARTITION p1 VALUES LESS THAN (3000),
           PARTITION p2 VALUES LESS THAN (4000),
           PARTITION p3 VALUES LESS THAN MAXVALUE
)
```

- Tämä ohjaisi tietueet palkan mukaan neljään eri osioon.
- Esimerkiksi lokidataa sisältävässä taulussa tietueet voitaisiin osioida aikaleiman perusteella.
  - Vanhan osion poisto olisi nyt nopeaa **ALTER TABLE … DROP PARTITION** -lauseella (vrt. **DELETE-lause**).
- Ositus voidaan tehdä myös hajautusarvon perusteella.
  - Tällöin kentästä lasketaan hajautin, joka määrää mihin osioon tietue ohjautuu.
  - Tuottaa (lähes) tasaisen määrän tietueita kaikkiin osioihin.

Kyselysuunnitelman tarkastelu

- Miten saadaan selville, miten tiedonhallintaohjelmisto kyselyn suorittaa?
- **SELECT**-lauseen eteen asetetaan **EXPLAIN**-verbi.

```
EXPLAIN SELECT *
FROM TYONTEKIJA
WHERE Tyontekijanumero=13;
```

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	SIMPLE	TYONTEKIJA	const	PRIMARY	PRIMARY	4	const	1	

Kyselysuunnitelman tarkastelu

- Ilman indeksiä:

```
EXPLAIN SELECT *
FROM TYONTEKIJA
WHERE Sukunimi="Virtanen";
```

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	SIMPLE	TYONTEKIJA	ALL	(NULL)	(NULL)	(NULL)	(NULL)	200 524	Using where

Paljastaa peräkkäishaun

- Sama sukunimen mukaan indeksoituna:

```
EXPLAIN SELECT *
FROM TYONTEKIJA
WHERE Sukunimi="Virtanen";
```

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	SIMPLE	TYONTEKIJA	ref	idx_sukunimi	idx_sukunimi	42	const	500	Using index condition

Suorituskykysimulaatiot

- Yksittäinen kokeilukerta ei yleensä anna kattavaa kuvaa tietokantapalvelimen suorituskyvystä ja pullonkauloista.
- Yksittäisen kyselyn vasteaikaan voivat vaikuttaa:
  - Tietokantapalvelimen kuormitus (aktiivisten tietokantayhteyksien määrä)
  - Mahdolliset välimuistit
  - Kyselyssä olevat parametrien arvot
- Parempi käsitys saadaan simuloimalla tietokantapalvelimeen kohdistuvaa kuormitusta.
- Simuloinnilla voit testata esimerkiksi indeksoinnin vaikutusta ja palvelimen skaalautuvuuskapasiteettia.

Metropolia

Ennen indeksointia...

```
C:\Windows\System32>mysqlslap --user=root --password --concurrency=20 --iterations=100
 --create-schema=bigcompany --query="Z:\temp\queries.sql"
Enter password: **********
Benchmark
        Average number of seconds to run all queries: 1.016 seconds
        Minimum number of seconds to run all queries: 0.422 seconds
        Maximum number of seconds to run all queries: 1.609 seconds
        Number of clients running queries: 20
        Average number of queries per client: 5
```

queries.sql

```
SELECT lastname, firstname FROM employee WHERE firstname = "Minna";
SELECT lastname, firstname FROM employee WHERE firstname = "Uuno";
SELECT lastname, firstname FROM employee WHERE firstname = "Urpo";
SELECT lastname, firstname FROM employee WHERE firstname = "Matleena";
SELECT lastname, firstname FROM employee WHERE firstname = "Lyly";
```

- mysqlslap-komento annetaan komentokehotteessa, jonka avaat Windows-käyttöjärjestelmässä esim. seuraavasti:
  - Paina Windows-painiketta
  - Ala kirjoittaa ”Command prompt” ja valitse ”Command Prompt (MariaDB 11.5. (x64))” tai vastaava.

Metropolia

Indeksoinnin jälkeen...

```
CREATE INDEX idx_lastname ON employee(lastname);
```

```
C:\Windows\System32>mysqlslap --user=root --password --concurrency=20 --iterations=100
 --create-schema=bigcompany --query="Z:\temp\queries.sql"
Enter password: **********
Benchmark
        Average number of seconds to run all queries: 0.075 seconds
        Minimum number of seconds to run all queries: 0.015 seconds
        Maximum number of seconds to run all queries: 1.047 seconds
        Number of clients running queries: 20
        Average number of queries per client: 5
```

- Kun teet muutoksen tietokantaan (eli tässä tapauksessa indeksoinnin), pidä simulaatioparametrit samoina. Ne olivat:
  - Samanaikaisten yhteyksien määrä (20)
  - Toistokertojen lukumäärä (100), pienentää tuloksen satunnaisvaihtelua
  - Kyselytiedosto (queries.sql)
- Näet, että keskimääräinen suoritusaika oli pudonnut indeksoinnin vaikutuksesta 1,016 sekunnista 0,075 sekuntiin.

Metropolia